基于几何光学的教室黑板镜面反射模型与有限俯仰角下的最优调节方案

王瑞德　王家件　胡益玮
广州大学附属中学　高二

2026 年 4 月

摘要

教室黑板与一体机屏幕在强光下易产生镜面反射，导致局部座位看不清板书。本文在单侧窗户平行光入射、黑板可绕底边小角度俯仰的设定下，先建立竖直截面内的反射模型，再在此基础上将座位区抽象为水平面上的长方形区域，用离散网格判定「反射光与视线是否共线（在阈值角内）」以刻画受害范围。对每个授课时刻，在最大俯仰角不超过 $\theta_{\max}$ 的硬约束下，用一维扫描求使前排代表座位消除眩光的**最小必要**俯仰角 $\theta^*(t)$。数值仿真表明：在示意参数下，早读后段（如 8:00）受害座位面积比例可由约 11.6% 降至约 4.3%，并给出可调铰链与限位角的改造建议。

关键词：镜面反射；黑板俯仰；眩光；离散优化；教室采光

1　问题重述

日常教学中，前排同学常因黑板局部反光看不清板书，一体机玻璃屏尤为明显。光源主要为日光与灯光的镜面反射。拉窗帘、调座位、换哑光板各有代价。更根本的做法是**略微改变黑板法向**，使反射光偏离学生视域。本题要求：建立几何光学模型，求**有限转角**下的最优（本文取「最小必要」）俯仰方案，并给出可落地改造思路。

2　模型假设

1. 以**平行光**近似太阳光；主光源从教室**单侧窗户**入射，方向随时间缓慢变化。
2. 黑板（含一体机显示区中的镜面分量）抽象为**平面镜面**，绕与黑板底边重合的水平轴俯仰，整板同角 θ。
3. 座位区在水平面内为**长方形**；学生眼高取统一值 h_e（坐姿统计均值）。

4. 当黑板某采样点处反射光方向与「该生看向该点」的视线夹角小于阈值 ε 时，判为**受害**（眩光显著影响辨认）。

5. 忽略多次反射与散射的次要贡献；不讨论偏振与色散。

3 符号说明

符号	含义
θ	黑板俯仰角（相对竖直位置，单位 °）
$\theta_{\max}$	允许的最大俯仰角（书写与观看约束）
$\theta^*(t)$	时刻 t 的最小必要俯仰角
$\mathbf{n}$	黑板外法向（指向教室一侧）
$\mathbf{l}$	入射光方向（指向板面，满足 $\mathbf{l}\cdot\mathbf{n}<0$）
$\mathbf{r}$	反射光方向（自板面射出）
ε	眩光判定半角阈值

4 模型建立

4.1 反射定律（向量形式）

设 $\mathbf{l}$、$\mathbf{n}$ 均为单位向量，$\mathbf{n}$ 为外法向。反射方向

$$\mathbf{r}=\mathbf{l}-2(\mathbf{l}\cdot\mathbf{n})\mathbf{n}. \tag{1}$$

4.2 竖直截面几何（板书推导主模型）

取含黑板法向与窗户主入射方向的**竖直平面**。黑板底边离地 z_{bot}，板高 H，俯仰后板面法向随 θ 变化。太阳方向在该平面内（或由三维方向投影）记为随时间变化的入射单位向量。

4.3 长方形教室与座位受害判定（俯视展示）

建立右手系：X 轴垂直黑板指向教室内侧，Y 轴沿黑板宽度方向，Z 轴竖直向上。黑板底边过点 $(0,y_{\min},z_{\text{bot}})$，板面离散为 $n_u\times n_v$ 个采样点 $\{P_k\}$。座位平面 $z=h_e$，在 $[x_{\min},x_{\max}]\times[y_{\min},y_{\max}]$ 上网格化，网格中心为眼睛位置 E_{ij}。

对固定 (θ,t)，取该时刻入射方向 $\mathbf{l}(t)$。若存在某 P_k 使

$$\angle\big(\mathbf{r}_k,\ \overrightarrow{P_kE_{ij}}\big)<\varepsilon,\qquad \mathbf{r}_k=\mathbf{l}-2(\mathbf{l}\cdot\mathbf{n})\mathbf{n}, \tag{2}$$

则座位 (i, j) 判为受害。

4.4 逐时刻优化（最小必要角）

对前排**代表座位** E_0（本文取近第一排、$y = 0$ 处），离散扫描 $\theta \in [0, \theta_{\max}]$（步长如 0.05°），取**第一个**使 E_0 不受害的 θ 作为 $\theta^*(t)$，体现「尽量少动黑板」。若全程受害，则取 $\theta_{\max}$ 并标记为不可行（可配合窗帘等辅助措施）。

5 模型求解与算法

1. 输入教室与黑板几何、ε、$\theta_{\max}$、时间离散 $\{t_m\}$。
2. 对每个 t_m，生成 $\mathbf{l}(t_m)$（本文用水平方位角随授课时段线性扫过、仰角缓变的示意函数模拟广州地区春秋分附近晴天侧窗工况）。
3. 一维扫描得 $\theta^*(t_m)$。
4. 对选定时刻，计算 $\theta = 0$ 与 $\theta = \theta^*$ 下座位网格受害二值图，输出面积比例 ρ_0, ρ_*。

程序使用 Python（NumPy/Matplotlib）实现，完整源代码已附于附录。

6 结果与分析

6.1 逐时刻最小必要俯仰角

图 **??** 给出 8:00–17:00、步长 0.5 h 的 $\theta^*(t)$。可见在示意入射角设定下，清晨若干时刻需要 1°–3° 量级的调节，其余时段可与竖直位置接近。

6.2 受害座位区域：调节前后对比（长方形教室）

在**同一时刻、同一太阳方向**下，图 **??** 对比 $\theta = 0$ 与 $\theta = \theta^*$ 的俯视填色图（红色表示该格中心座位受害）。数值上，本组示意参数在 $t = 8{:}00$ 时，受害格点比例由约 11.6% 降至约 4.3%。

6.3 竖直截面光路示意

图 **??** 给出最坏示意时刻下，调节前后入射方向、法向与反射方向在 xz 截面内的关系（箭头长度仅作示意）。

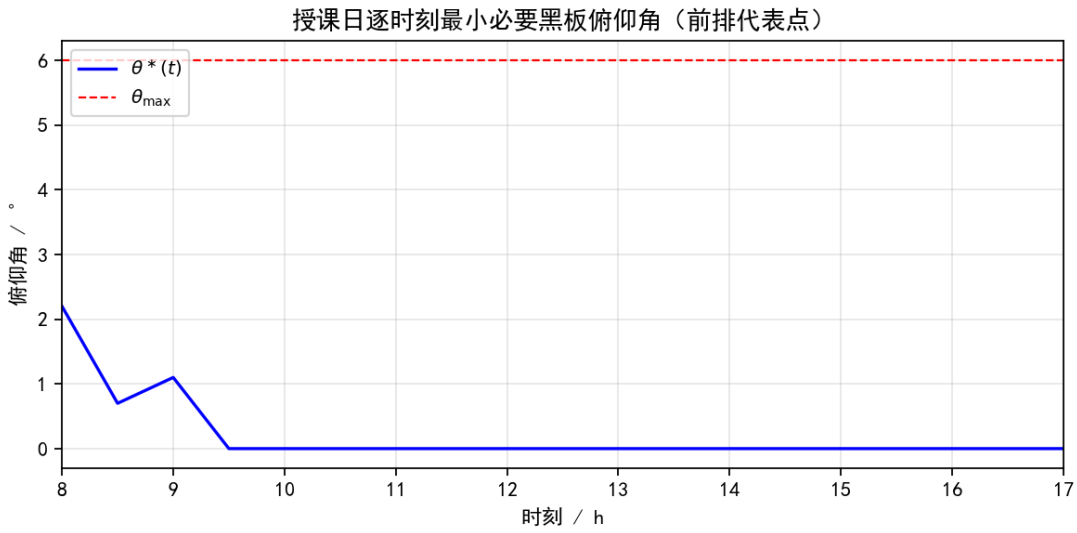

图 1: 授课日逐时刻最小必要黑板俯仰角（前排代表点）与 $\theta_{\max}$ 上界示意

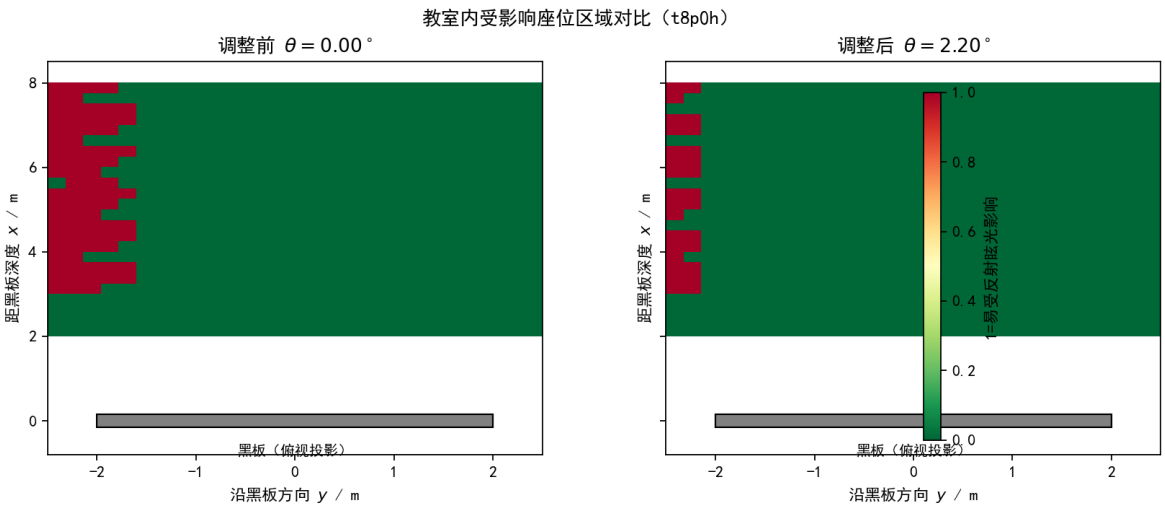

图 2: $t = 8{:}00$ 示意：调节前后受害座位区域对比（俯视）

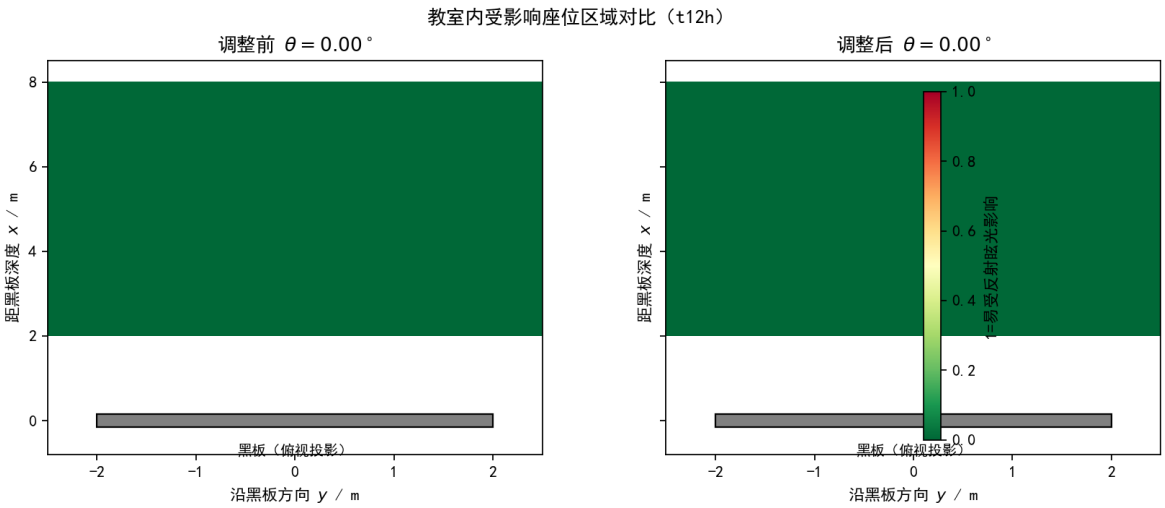

图 3: $t = 12{:}00$ 示意：调节前后对比（受害区域缩小程度因入射方向而异）

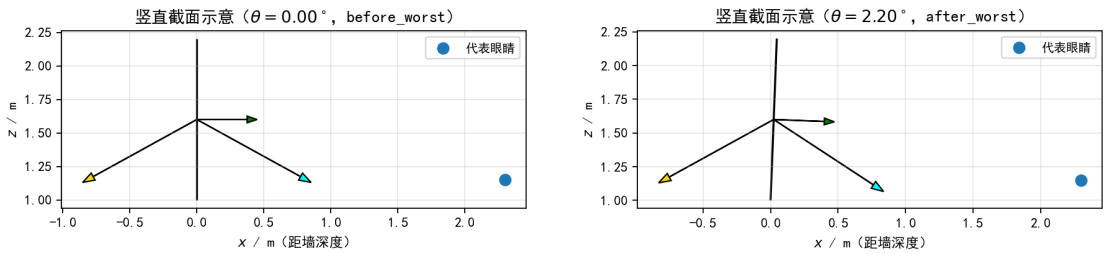

图 4: 竖直截面光路示意：调节前（左）与调节后（右）

7 灵敏度分析

θ^* 与受害比例对以下因素敏感：阈值 ε、$\theta_{\max}$、窗户朝向与教室朝向组合、眼高 h_e、第一排距墙距离。提高 ε 会扩大「受害」判定（更保守）；增大 $\theta_{\max}$ 可扩大可行域，但削弱书写舒适性约束的实际意义。

8 改造方案（可落地）

- **结构**：黑板（或推拉绿板整体）下沿安装**可调铰链**，上沿或背部设**限位拉杆**，标定 $0°$–$\theta_{\max}$ 刻度（建议 $\theta_{\max} \in [3°, 8°]$ 由校方与厂商共定）。
- **使用规程**：早读、上午侧窗强光时段按示意图或简易日照表微调；平时回零。
- **协同**：极端角度仍不足时，优先**百叶/半帘**削弱平行光而非全遮光。

9 模型评价

优点：几何清晰，易于程序实现与可视化；「最小必要角」贴合教学场景。
局限：平行光与统一眼高为近似；未单独拆分一体机贴膜与粉笔板漫反射差异。
推广：可加入多扇窗离散光源、多排不同眼高、或目标改为「最小化受害座位数」的加权优化。

参考文献

1. Hecht E. Optics (相关镜面反射与成像章节).
2. 建筑采光设计标准（GB 相关条文，可按实际版本引用）.

附录 A：仿真程序（Python 全文）

本文插图由下列程序生成。依赖：`numpy`、`matplotlib`。将代码保存为 `simulate.py` 后,在工程根目录执行 `python simulate.py`,可在 `figures/` 下得到正文插图及 `summary.json`。以下与仓库中 `code/simulate.py` 一致，便于仅提交 PDF 时仍保留可复现依据。

```
""" 黑板镜面反射与最优俯仰角 —数值仿真与作图（修正入射方向约定）"""
from __future__ import annotations

import json
from pathlib import Path
```

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle
from matplotlib import rcParams

rcParams["font.sans-serif"] = ["SimHei", "Microsoft YaHei", "DejaVu Sans"]
rcParams["axes.unicode_minus"] = False

ROOT = Path(__file__).resolve().parent.parent
FIG_DIR = ROOT / "figures"
FIG_DIR.mkdir(parents=True, exist_ok=True)


class Params:
    z_bot = 1.0
    H = 1.2
    y_board_min = -2.0
    y_board_max = 2.0
    x_seat_min = 2.0
    x_seat_max = 8.0
    y_seat_min = -2.5
    y_seat_max = 2.5
    z_eye = 1.15
    epsilon_deg = 9.0
    theta_max_deg = 6.0
    n_u = 8
    n_v = 6
    nx_grid = 28
    ny_grid = 24


P = Params()


def board_frame(theta_deg: float):
    th = np.radians(theta_deg)
    H = P.H
    T = np.array([H * np.sin(th), 0.0, H * np.cos(th)], dtype=float)
    Wvec = np.array([0.0, P.y_board_max - P.y_board_min, 0.0])
    n = np.cross(Wvec, T)
    n = n / np.linalg.norm(n)
    u_hat = T / np.linalg.norm(T)
    O = np.array([0.0, P.y_board_min, P.z_bot])
    return n, O, u_hat, H
```

```
def sample_board_points(theta_deg: float):
    n, O, u_hat, H = board_frame(theta_deg)
    pts = []
    for ui in np.linspace(0.0, 1.0, P.n_u):
        for vi in np.linspace(0.0, 1.0, P.n_v):
            p = O + ui * H * u_hat + vi * (P.y_board_max - P.y_board_min) *
            ↪  np.array([0.0, 1.0, 0.0])
            pts.append(p)
    return np.array(pts), n


def reflect(incident_to_surface: np.ndarray, n_out: np.ndarray) -> np.ndarray:
    """ 入射方向指向板面，外法向 n_out 指向教室（+X 侧）；满足 dot(l,n_out)<0。"""
    l = incident_to_surface / np.linalg.norm(incident_to_surface)
    r = l - 2.0 * np.dot(l, n_out) * n_out
    return r / np.linalg.norm(r)


def sun_incident_direction(t_hours: float) -> np.ndarray:
    """
    单侧窗平行光：传播方向指向黑板（需 l·n<0，n +X），随时刻水平方位变化。
    """
    frac = np.clip((t_hours - 8.0) / 9.0, 0.0, 1.0)
    az = np.radians(15.0 + 130.0 * frac)
    el = np.radians(28.0 + 10.0 * frac)
    cx, sx = np.cos(az), np.sin(az)
    ce, se = np.cos(el), np.sin(el)
    prop = np.array([-ce * cx, ce * sx, -se])
    prop = prop / np.linalg.norm(prop)
    return prop


def seat_glare(eye: np.ndarray, theta_deg: float, l_in: np.ndarray) -> bool:
    pts, n = sample_board_points(theta_deg)
    eps = np.radians(P.epsilon_deg)
    if np.dot(l_in, n) >= 0:
        return False
    for p in pts:
        r = reflect(l_in, n)
        w = eye - p
        wn = np.linalg.norm(w)
        if wn < 1e-6:
            continue
        v = w / wn
        c = float(np.clip(np.dot(r, v), -1.0, 1.0))
```

```
        if float(np.arccos(c)) < eps:
            return True
    return False


def min_theta_for_seat(eye: np.ndarray, l_in: np.ndarray):
    for th in np.arange(0.0, P.theta_max_deg + 1e-9, 0.05):
        if not seat_glare(eye, float(th), l_in):
            return float(th), True
    return float(P.theta_max_deg), False


def affected_mask(theta_deg: float, l_in: np.ndarray):
    xs = np.linspace(P.x_seat_min, P.x_seat_max, P.nx_grid)
    ys = np.linspace(P.y_seat_min, P.y_seat_max, P.ny_grid)
    Xc, Yc = np.meshgrid(xs, ys, indexing="ij")
    mask = np.zeros_like(Xc, dtype=bool)
    for i in range(P.nx_grid):
        for j in range(P.ny_grid):
            eye = np.array([Xc[i, j], Yc[i, j], P.z_eye])
            mask[i, j] = seat_glare(eye, theta_deg, l_in)
    return Xc, Yc, mask


def plot_topdown_compare(l_in: np.ndarray, theta0: float, theta1: float, tag: str):
    _, _, m0 = affected_mask(theta0, l_in)
    _, _, m1 = affected_mask(theta1, l_in)
    extent = [P.y_seat_min, P.y_seat_max, P.x_seat_min, P.x_seat_max]
    fig, axes = plt.subplots(1, 2, figsize=(10.5, 4.6), sharey=True)
    im = None
    for ax, m, title in zip(
        axes,
        [m0, m1],
        [rf"调整前 $\theta={theta0:.2f}^\circ$", rf"调整后
        ↪  $\theta={theta1:.2f}^\circ$"],
    ):
        im = ax.imshow(
            m.T.astype(float),
            origin="lower",
            extent=extent,
            aspect="auto",
            cmap="RdYlGn_r",
            vmin=0,
            vmax=1,
            interpolation="nearest",
```

```
        )
        ax.set_xlabel(" 沿黑板方向 $y$ / m")
        ax.set_ylabel(" 距黑板深度 $x$ / m")
        ax.set_title(title)
        ax.add_patch(
            Rectangle(
                (P.y_board_min, -0.15),
                P.y_board_max - P.y_board_min,
                0.3,
                facecolor="gray",
                edgecolor="k",
            )
        )
        ax.text(0.5 * (P.y_board_min + P.y_board_max), -0.55, "黑板（俯视投影）",
        ↪  ha="center", va="top", fontsize=9)
        ax.set_ylim(-0.8, P.x_seat_max + 0.5)
    fig.colorbar(im, ax=axes.ravel().tolist(), label="1=易受反射眩光影响",
    ↪  shrink=0.85)
    fig.suptitle(f" 教室内受影响座位区域对比（{tag}）", fontsize=12)
    fig.subplots_adjust(left=0.06, right=0.98, top=0.88, bottom=0.14, wspace=0.25)
    out = FIG_DIR / f"topdown_compare_{tag}.png"
    fig.savefig(out, dpi=160)
    plt.close(fig)
    return out, float(m0.mean()), float(m1.mean())


def plot_theta_curve(times: np.ndarray, thetas: np.ndarray, feas: np.ndarray):
    fig, ax = plt.subplots(figsize=(7.5, 3.8))
    ax.plot(times, thetas, "b-", lw=1.5, label=r"$\theta^\ast(t)$")
    ax.axhline(P.theta_max_deg, color="r", ls="--", lw=1, label=r"$\theta_{\max}$")
    bad = ~feas
    if np.any(bad):
        ax.scatter(times[bad], thetas[bad], c="orange", s=22, zorder=5,
        ↪  label="上限内未完全消除（示意）")
    ax.set_xlabel(" 时刻 / h")
    ax.set_ylabel(r" 俯仰角 / $^\circ$")
    ax.set_title(" 授课日逐时刻最小必要黑板俯仰角（前排代表点）")
    ax.legend(loc="upper left", fontsize=9)
    ax.set_xlim(times.min(), times.max())
    ax.grid(True, alpha=0.3)
    fig.tight_layout()
    fig.savefig(FIG_DIR / "theta_of_t.png", dpi=160)
    plt.close(fig)
```

```
def plot_section(theta_deg: float, l_in: np.ndarray, eye_xz: tuple[float, float],
↪  tag: str):
    th = np.radians(theta_deg)
    H = P.H
    z1 = P.z_bot + H * np.cos(th)
    x1 = H * np.sin(th)
    fig, ax = plt.subplots(figsize=(5.5, 4.2))
    ax.fill([0, x1, x1, 0], [P.z_bot, z1, z1, P.z_bot], color="#555", edgecolor="k",
    ↪  lw=1.5)
    ex, ez = eye_xz
    ax.plot(ex, ez, "o", color="C0", ms=8, label=" 代表眼睛")
    n, _, _, _ = board_frame(theta_deg)
    midx, midz = 0.5 * x1, 0.5 * (P.z_bot + z1)
    scale = 1.0
    ax.arrow(midx, midz, scale * l_in[0], scale * l_in[2], head_width=0.06,
    ↪  fc="gold", ec="k", length_includes_head=True)
    ax.arrow(midx, midz, 0.45 * n[0], 0.45 * n[2], head_width=0.05, fc="g", ec="k",
    ↪  length_includes_head=True)
    r = reflect(l_in, n)
    ax.arrow(midx, midz, scale * r[0], scale * r[2], head_width=0.06, fc="cyan",
    ↪  ec="k", length_includes_head=True)
    ax.set_xlabel("$x$ / m（距墙深度）")
    ax.set_ylabel("$z$ / m")
    ax.set_title(f" 竖直截面示意（$\\theta={theta_deg:.2f}^\\circ$, {tag}）")
    ax.set_aspect("equal", adjustable="box")
    ax.legend(loc="upper right", fontsize=9)
    ax.grid(True, alpha=0.3)
    fig.tight_layout()
    fig.savefig(FIG_DIR / f"section_{tag}.png", dpi=160)
    plt.close(fig)


def main():
    times = np.arange(8.0, 17.01, 0.5)
    eye_ref = np.array([P.x_seat_min + 0.3, 0.0, P.z_eye])
    thetas_star, feas, rows = [], [], []
    for t in times:
        l_in = sun_incident_direction(t)
        th_star, ok = min_theta_for_seat(eye_ref, l_in)
        thetas_star.append(th_star)
        feas.append(ok)
        _, _, m0 = affected_mask(0.0, l_in)
        _, _, m1 = affected_mask(th_star, l_in)
        rows.append(
            {
```

```
                "t_h": float(t),
                "theta_star_deg": float(th_star),
                "feasible": bool(ok),
                "affected_frac_theta0": float(m0.mean()),
                "affected_frac_opt": float(m1.mean()),
            }
        )
    thetas_star = np.array(thetas_star)
    feas = np.array(feas)
    plot_theta_curve(times, thetas_star, feas)
    worst_idx = int(np.argmax([r["affected_frac_theta0"] for r in rows]))
    t_w = rows[worst_idx]["t_h"]
    l_w = sun_incident_direction(t_w)
    th_w = rows[worst_idx]["theta_star_deg"]
    plot_topdown_compare(l_w, 0.0, th_w, f"t{t_w:.1f}h".replace(".", "p"))
    idx12 = int(np.where(np.isclose(times, 12.0))[0][0])
    th_n = rows[idx12]["theta_star_deg"]
    plot_topdown_compare(sun_incident_direction(12.0), 0.0, th_n, "t12h")
    ex, ez = float(eye_ref[0]), float(eye_ref[2])
    plot_section(0.0, l_w, (ex, ez), "before_worst")
    plot_section(th_w, l_w, (ex, ez), "after_worst")
    (FIG_DIR / "summary.json").write_text(json.dumps({"series": rows},
    ↪  ensure_ascii=False, indent=2), encoding="utf-8")
    print(" 已写入:", FIG_DIR)
    w = rows[worst_idx]
    print("最坏时刻 t=", w["t_h"], "theta*=", w["theta_star_deg"], "受害比例",
    ↪  w["affected_frac_theta0"], "->", w["affected_frac_opt"])


if __name__ == "__main__":
    main()
```